

Spring Core – Day 11 Notes

Setter & Constructor Injection and Bean Autowiring (byName, byType, constructor)

1. Setter and Constructor Injection – Recap

Consider two classes, **A** and **B**, where class A holds a reference to class B and injects it through a setter method.

```
class A {
    B b;

    public void set(B b) {
        this.b = b;
    }
}

class B {
}
```

application-context.xml

```
<beans>

    <bean id="First" class="packageName.A">
        <!--
            This setter injection and constructor injection is currently
            done manually by the programmer.
            We want this injection to be taken care of by the IOC container.
        -->
        <property name="b" ref="sec"></property>

        <constructor-arg ref="sec"></constructor-arg>
    </bean>

    <bean id="sec" class="packageName.B">
    </bean>

</beans>
```

2. Bean Autowiring

Instead of declaring the dependency between bean definitions ourselves, the Spring IOC container can take care of identifying the dependencies and performing the injection automatically. This is done using **bean autowiring**.

By default, autowiring is **disabled**.

How to enable autowiring?

We have to write an attribute inside the bean tag: **autowire="mode"**

Syntax

```
<bean id="    " class="    " autowire="mode"></bean>
```

This attribute indicates two things:

- How to identify the dependency
- How to inject the dependency

The possible values for the mode are:

- byName
- byType
- constructor

Example – Robot and Chip

```
class Robot {

    Chip chip;

    public void setChip(Chip chip) {
        this.chip = chip;
    }

    public String toString() {
        return "Robot [chip=" + chip + "]";
    }
}

class Chip {

    String producer;
    String power;

    public void setProducer(String producer) {
        this.producer = producer;
    }

    public void setPower(String power) {
        this.power = power;
    }

    public String toString() {
        return "Chip [producer=" + producer + ", power=" + power + "]";
    }
}
```

application-context.xml

```
<beans>

    <bean id="chip" class="packageName.Chip">
        <property name="producer" value="NVIDIA"/>
        <property name="power" value="1231"/>
    </bean>

    <bean id="robot" class="packageName.Robot" autowire="byName">
        <property name="chip" ref="chip"/>
        <!--
            This ref injection is what we do NOT want to write ourselves.
            We want the IOC container to inject this automatically.
            That is why we write autowire="mode" inside the bean tag.
        -->
    </bean>

</beans>
```

When we provide **autowire="byName"**, the IOC container immediately looks at the class for which the bean is declared and checks the attribute name present inside that class. After picking up that attribute name, it searches for a bean definition in the configuration file with the same name as the attribute. If found, it injects the dependency with the help of the setter method.

2.1 byName

- Identify the dependency by matching the attribute name of the target class with a bean id in the IOC container.
- Perform the injection of the dependency through the setter method.

Note: If we change the attribute name of the class and try to inject with a different name, we get a **null value**, not an exception.

application-context.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           https://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="chip" class="com.example.Chip">
        <property name="producer" value="NVIDIA"></property>
        <property name="power" value="1234"></property>
    </bean>

    <bean id="robot" class="com.example.Robot" autowire="byName">
        <!-- <property name="chip" ref="chip"></property> -->
    </bean>

</beans>
```

2.2 byType

- Identify the dependency by matching the class type of the target class attribute with the bean's class type in the IOC container.
- Perform the injection of the dependency through the setter method.

Both **byName** and **byType** work through the setter method. The basic difference between them is:

- In **byType**, if we change the name of the class attribute, there is no issue in injecting the value, because it checks the type, not the name.
- In **byName**, if we change the name of the class attribute, the value will not get injected, because it works with the name, not the type.

Note: If we use **byType** and provide two beans of the same type, we get an exception, because the IOC container gets confused about which bean should be injected. To resolve this, we write **autowire-candidate="false"** on one of the beans.

application-context.xml

```
<bean id="chip" class="com.example.Chip" autowire-candidate="false">
    <property name="producer" value="NVIDIA"></property>
    <property name="power" value="1234"></property>
</bean>

<bean id="chip1" class="com.example.Chip">
    <property name="producer" value="Intel"></property>
    <property name="power" value="5555"></property>
```

```

</bean>

<bean id="robot" class="com.example.Robot" autowire="byType">
    <!-- <property name="chip" ref="chip"></property> -->
</bean>

```

2.3 constructor

- Identify the dependency by matching the constructor argument type of the target class.
- Perform the injection through constructor injection.

Note: If we add two beans of the same class type while using autowire="constructor", we do **not** get an exception, but the value also does not get injected, because both beans are of the same type present in the constructor parameter. To get the value injected, we must write **autowire-candidate="false"** on one of the bean definitions.

Note: If a class has multiple constructors and we use constructor autowiring, the IOC container injects the dependency through constructor injection by invoking the constructor with the maximum number of parameters that it is able to satisfy.

3. Full Example – Robot, Chip and Sensor

application-context.xml

```

<bean id="sensor" class="com.example.Sensor">
    <constructor-arg value="1"></constructor-arg>
    <constructor-arg value="true"></constructor-arg>
</bean>

<bean id="chip" class="com.example.Chip" autowire-candidate="false">
    <constructor-arg value="NVIDIA"></constructor-arg>
    <constructor-arg value="12123"></constructor-arg>
</bean>

<bean id="chip1" class="com.example.Chip">
    <constructor-arg value="Intel"></constructor-arg>
    <constructor-arg value="55555"></constructor-arg>
</bean>

<bean id="robot" class="com.example.Robot" autowire="constructor">
</bean>

```

Robot.java

```

package com.example;

public class Robot {

    private Chip chip;
    private Sensor sensor;

    public Robot() {
        System.out.println("0 param");
    }

    public Robot(Chip chip) {
        System.out.println("One param");
        this.chip = chip;
    }

    public Robot(Chip chip, Sensor sensor) {
        System.out.println("2 param");
    }
}

```

```

        this.chip = chip;
        this.sensor = sensor;
    }

    public Chip getChip() {
        return chip;
    }

    public void setChip(Chip chip) {
        this.chip = chip;
    }

    public Sensor getSensor() {
        return sensor;
    }

    public void setSensor(Sensor sensor) {
        this.sensor = sensor;
    }

    @Override
    public String toString() {
        return "Robot [chip=" + chip + ", sensor=" + sensor + "]";
    }
}

```

Chip.java

```

package com.example;

public class Chip {

    private String producer;
    private String power;

    public Chip() {
    }

    public Chip(String producer, String power) {
        this.producer = producer;
        this.power = power;
    }

    public String getProducer() {
        return producer;
    }

    public void setProducer(String producer) {
        this.producer = producer;
    }

    public String getPower() {
        return power;
    }

    public void setPower(String power) {
        this.power = power;
    }

    @Override
    public String toString() {
        return "Chip [producer=" + producer + ", power=" + power + "]";
    }
}

```

Sensor.java

```

package com.example;

public class Sensor {

    private int reflectionTime;
    private boolean isActivate;

    public Sensor() {
    }

    public Sensor(int reflectionTime, boolean isActivate) {
        this.reflectionTime = reflectionTime;
        this.isActivate = isActivate;
    }

    public int getReflectionTime() {
        return reflectionTime;
    }

    public void setReflectionTime(int reflectionTime) {
        this.reflectionTime = reflectionTime;
    }

    public boolean isActivate() {
        return isActivate;
    }

    public void setActivate(boolean isActivate) {
        this.isActivate = isActivate;
    }

    @Override
    public String toString() {
        return "Sensor [reflectionTime=" + reflectionTime + ", isActivate=" + isActivate + "];"
    }
}

```

application-context.xml (complete)

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           https://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="sensor" class="com.example.Sensor">
        <constructor-arg value="1"></constructor-arg>
        <constructor-arg value="true"></constructor-arg>
    </bean>

    <!--
    <bean id="chip" class="com.example.Chip" autowire-candidate="false">
        <property name="producer" value="NVIDIA"></property>
        <property name="power" value="1234"></property>
    </bean>

    <bean id="chip1" class="com.example.Chip">
        <property name="producer" value="Intel"></property>
        <property name="power" value="5555"></property>
    </bean>

    <bean id="robot" class="com.example.Robot" autowire="byType">
        <property name="chip" ref="chip"></property>
    </bean>
-->

```

```

    <bean id="chip" class="com.example.Chip" autowire-candidate="false">
        <constructor-arg value="NVIDIA"></constructor-arg>
        <constructor-arg value="12123"></constructor-arg>
    </bean>

    <bean id="chip1" class="com.example.Chip">
        <constructor-arg value="Intel"></constructor-arg>
        <constructor-arg value="55555"></constructor-arg>
    </bean>

    <bean id="robot" class="com.example.Robot" autowire="constructor">
    </bean>

</beans>

```

Main.java

```

package com.example;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class Main {

    public static void main(String[] args) {

        ApplicationContext context =
            new ClassPathXmlApplicationContext("com/example/application-context.xml");

        Robot bean = context.getBean("robot", Robot.class);

        System.out.println(bean);
    }
}

```